Sesión 21: Clean Architecture en Flutter (Estructuración por Capas: Data [Repositorios/DataSources], Domain [Entities/UseCases] y Presentation)

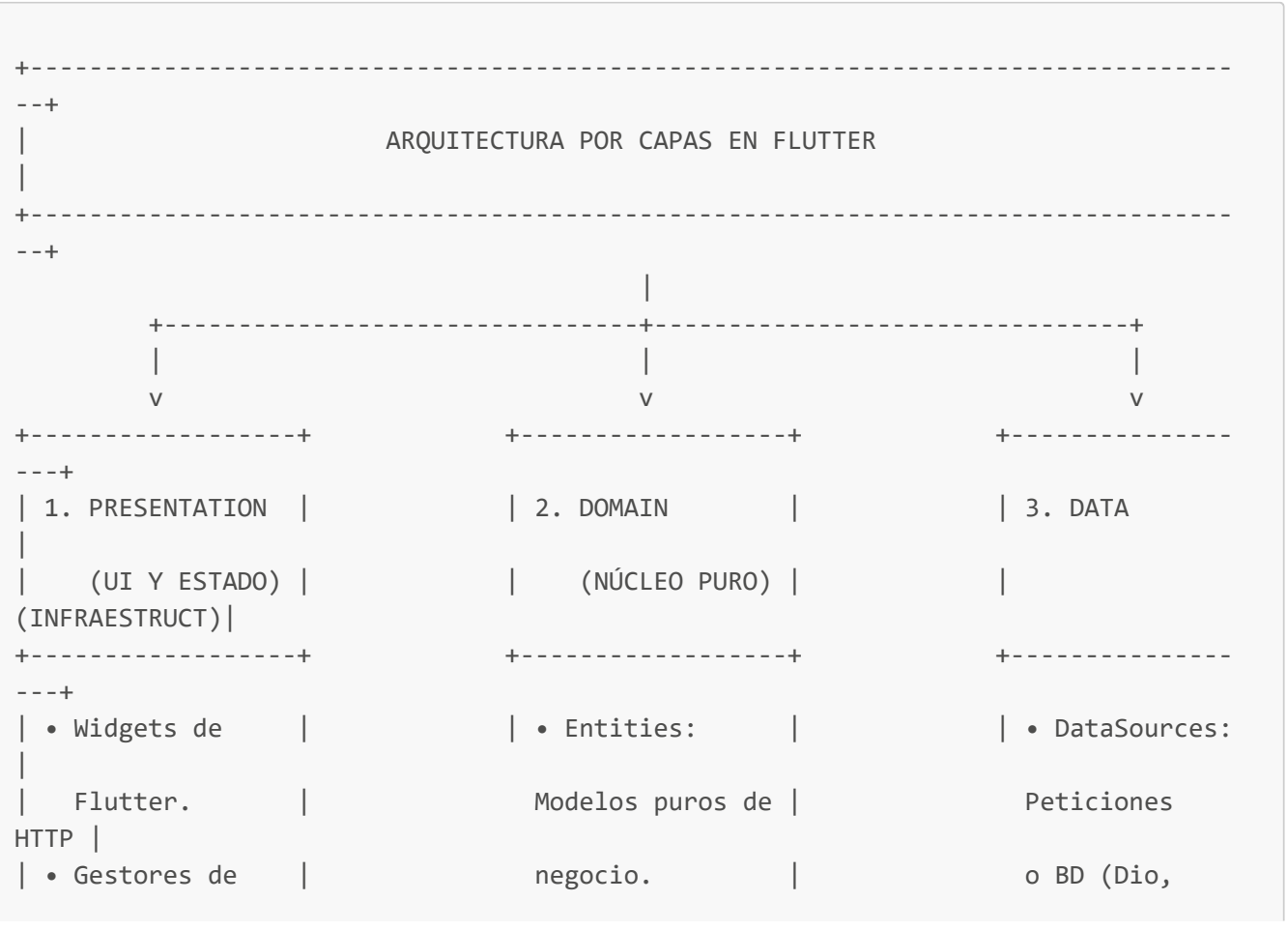
Introducción

A medida que las aplicaciones móviles crecen en escala y complejidad, mantener la separación de responsabilidades se vuelve indispensable para asegurar la mantenibilidad, escalabilidad y testabilidad del software. *Clean Architecture*, promovida por Robert C. Martin ("Uncle Bob"), es una disciplina de diseño que aísla la lógica central de negocio de las dependencias externas (marcos de trabajo, bibliotecas de UI, bases de datos o servicios API).

En esta sesión abordaremos la implementación de *Clean Architecture* en Flutter. Analizaremos la división en tres capas concéntricas esenciales: **Domain**, **Data** y **Presentation**, el principio de inversión de dependencias mediante interfaces abstractas, y cómo estructurar los casos de uso (*Use Cases*) para conectar el estado de la UI con las fuentes de datos.

Conceptos y Definiciones

El fundamento de *Clean Architecture* radica en la **Regla de Dependencia**: las capas internas no deben saber nada sobre las capas externas. La capa de Dominio es el núcleo puro de la aplicación y carece por completo de dependencias de Flutter o paquetes de terceros.



```
Isar)|
|   Estado (BLoC,  |             | • Use Cases:     |             | • Models:
|
|   Riverpod).     |               Reglas y casos   |               Mapeo de
JSON/BD|
| • Consume los    |               de uso puros.    |               con
toEntity().  |
|   Use Cases del  |             | • Repositories:  |             | • Repositories
|
|   Domain.        |               Contratos/Interfaz|               Impl:
Concreción |
+------------------+               abstractos.      |               de la
interfaz.  |
                                 +------------------+             +---------------
---+
```

1. Capa de Dominio (*Domain Layer*)

Es la capa más interna y aislada de la aplicación. No importa `flutter/material.dart` ni paquetes externos.

- **Entities:** Objetos de dominio puros que representan las reglas e identidades centrales del negocio.
- **Use Cases (Interactors):** Clases que encapsulan un flujo específico o regla de negocio (ej. `ObtenerListaAlumnos`, `AutenticarUsuario`).
- **Repository Contracts (Interfaces):** Clases abstractas que definen *qué* datos necesita el dominio sin especificar *cómo* se obtienen de la red o base de datos.

---

2. Capa de Datos (*Data Layer*)

Se encarga de la infraestructura y la persistencia de datos. Implementa los contratos definidos por el Dominio.

- **Data Sources:** Clases encargadas de interactuar directamente con fuentes externas (Remote DataSource para REST/GraphQL o Local DataSource para SQLite/Isar/Preferences).
- **Models:** Extensiones de las *Entities* que incluyen métodos de serialización/deserialización (`fromJson`, `toJson`, `fromMap`). Permiten desacoplar la API externa de la entidad pura.
- **Repository Implementations:** Clases concretas que implementan la interfaz del repositorio definida en el Dominio, decidiendo cuándo consultar el caché local o la red.

---

3. Capa de Presentación (*Presentation Layer*)

Es la capa encargada del renderizado visual y la interacción con el usuario.

- **UI Widgets:** Pantallas y componentes reutilizables de Flutter.
- **State Management:** BLoC, Riverpod, Notifiers o Cubit. Se comunican exclusivamente con los *Use Cases* del Dominio para obtener o mutar el estado de la aplicación, manteniendo los widgets libres de lógica de negocio.

Ejemplo Completo con Código Fuente: Consulta de Alumnos en ESCOM con Clean Architecture

El siguiente ejemplo implementa de forma integral la estructura de tres capas para consultar información académica, respetando la inversión de dependencias y la separación estricta de responsabilidades.

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';

void main() {
  // ============================================================================
  // INYECCIÓN DE DEPENDENCIAS Y ENSAMBLAJE (MANUAL DE ARQUITECTURA)
  // ============================================================================

  // 1. Instanciación de Infraestructura (Data Layer)
  final remoteDataSource = AlumnoRemoteDataSourceImpl();
  final repository = AlumnoRepositoryImpl(remoteDataSource: remoteDataSource);

  // 2. Instanciación del Caso de Uso (Domain Layer)
  final getAlumnoUseCase = GetAlumnoUseCase(repository: repository);

  // 3. Ejecución de la UI (Presentation Layer)
  runApp(DemoCleanArchitectureApp(getAlumnoUseCase: getAlumnoUseCase));
}

// ============================================================================
// CAPA 1: DOMINIO (DOMAIN LAYER) - NÚCLEO PURO Y REGLAS DE NEGOCIO
// ============================================================================

/// Entity: Objeto de Negocio Puro
class Alumno {
  final String boleta;
  final String nombre;
  final double promedio;

  const Alumno({
    required this.boleta,
    required this.nombre,
    required this.promedio,
  });

  bool get esRegular => promedio >= 6.0;
}

/// Contrato Abstracto del Repositorio (Inversión de Dependencias)
abstract class AlumnoRepository {
  Future<Alumno> obtenerAlumnoPorBoleta(String boleta);
}

/// Use Case: Caso de Uso Puro que ejecuta la regla de negocio
class GetAlumnoUseCase {
```

```
  final AlumnoRepository repository;

  const GetAlumnoUseCase({required this.repository});

  Future<Alumno> execute(String boleta) async {
    if (boleta.isEmpty || boleta.length < 10) {
      throw Exception('La boleta ingresada no cumple el formato institucional.');
    }
    return await repository.obtenerAlumnoPorBoleta(boleta);
  }
}

// ============================================================================
// CAPA 2: DATOS (DATA LAYER) - INFRAESTRUCTURA Y REPOSICIONES
// ============================================================================

/// Model: Mapeo de DTO (Data Transfer Object) con conversión a Entity
class AlumnoModel extends Alumno {
  const AlumnoModel({
    required super.boleta,
    required super.nombre,
    required super.promedio,
  });

  factory AlumnoModel.fromJson(Map<String, dynamic> json) {
    return AlumnoModel(
      boleta: json['boleta'] ?? '',
      nombre: json['nombre_completo'] ?? '',
      promedio: (json['promedio_general'] as num).toDouble(),
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'boleta': boleta,
      'nombre_completo': nombre,
      'promedio_general': promedio,
    };
  }
}

/// Interface de Data Source
abstract class AlumnoRemoteDataSource {
  Future<AlumnoModel> fetchAlumnoFromApi(String boleta);
}

/// Implementación Concreta de Remote Data Source
class AlumnoRemoteDataSourceImpl implements AlumnoRemoteDataSource {
  @override
  Future<AlumnoModel> fetchAlumnoFromApi(String boleta) async {
    // Simulación de petición HTTP (ej. Dio / http)
    await Future.delayed(const Duration(seconds: 2));

    if (boleta == '2026600001') {
```

```
      final jsonResponse = {
        'boleta': '2026600001',
        'nombre_completo': 'Alan Turing López',
        'promedio_general': 9.4,
      };
      return AlumnoModel.fromJson(jsonResponse);
    } else {
      throw Exception('Alumno no encontrado en la base de datos de ESCOM.');
    }
  }
}

/// Implementación Concreta del Repositorio
class AlumnoRepositoryImpl implements AlumnoRepository {
  final AlumnoRemoteDataSource remoteDataSource;

  AlumnoRepositoryImpl({required this.remoteDataSource});

  @override
  Future<Alumno> obtenerAlumnoPorBoleta(String boleta) async {
    // Convierte el Model de la capa Data a la Entity de la capa Domain
    final alumnoModel = await remoteDataSource.fetchAlumnoFromApi(boleta);
    return alumnoModel; // AlumnoModel extiende de Alumno (Entity)
  }
}

// ============================================================================
// CAPA 3: PRESENTACIÓN (PRESENTATION LAYER) - VISTAS Y ESTADO
// ============================================================================

class DemoCleanArchitectureApp extends StatelessWidget {
  final GetAlumnoUseCase getAlumnoUseCase;

  const DemoCleanArchitectureApp({
    super.key,
    required this.getAlumnoUseCase,
  });

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Clean Architecture',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: PantallaConsultaAlumno(getAlumnoUseCase: getAlumnoUseCase),
    );
  }
}
```

```
class PantallaConsultaAlumno extends StatefulWidget {
  final GetAlumnoUseCase getAlumnoUseCase;

  const PantallaConsultaAlumno({
    super.key,
    required this.getAlumnoUseCase,
  });

  @override
  State<PantallaConsultaAlumno> createState() => _PantallaConsultaAlumnoState();
}

class _PantallaConsultaAlumnoState extends State<PantallaConsultaAlumno> {
  final _boletaController = TextEditingController(text: '2026600001');
  bool _cargando = false;
  Alumno? _alumno;
  String? _error;

  Future<void> _consultarAlumno() async {
    setState(() {
      _cargando = true;
      _error = null;
      _alumno = null;
    });

    try {
      // La UI consume EXCLUSIVAMENTE el Use Case del Domain
      final resultado = await
widget.getAlumnoUseCase.execute(_boletaController.text);
      setState(() {
        _alumno = resultado;
      });
    } catch (e) {
      setState(() {
        _error = e.toString().replaceAll('Exception: ', '');
      });
    } finally {
      setState(() {
        _cargando = false;
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Consulta de Expediente (Clean Arch)'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: Padding(
        padding: const EdgeInsets.all(20.0),
        child: Column(
```

```
            children: [
              TextField(
                controller: _boletaController,
                decoration: const InputDecoration(
                  labelText: 'Número de Boleta ESCOM',
                  border: OutlineInputBorder(),
                  prefixIcon: Icon(Icons.badge),
                ),
              ),
              const SizedBox(height: 16),

              SizedBox(
                width: double.infinity,
                height: 48,
                child: ElevatedButton(
                  onPressed: _cargando ? null : _consultarAlumno,
                  child: _cargando
                      ? const SizedBox(
                          height: 20,
                          width: 20,
                          child: CircularProgressIndicator(strokeWidth: 2),
                        )
                      : const Text('Consultar Dominio'),
                ),
              ),
              const SizedBox(height: 24),

              if (_error != null)
                Card(
                  color: Colors.red.shade50,
                  child: Padding(
                    padding: const EdgeInsets.all(16.0),
                    child: Row(
                      children: [
                        const Icon(Icons.error_outline, color: Colors.red),
                        const SizedBox(width: 12),
                        Expanded(
                          child: Text(
                            _error!,
                            style: const TextStyle(color: Colors.red),
                          ),
                        ),
                      ],
                    ),
                  ),
                ),

              if (_alumno != null)
                Card(
                  elevation: 3,
                  child: Padding(
                    padding: const EdgeInsets.all(16.0),
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
```

```
                    children: [
                      ListTile(
                        leading: const CircleAvatar(child: Icon(Icons.person)),
                        title: Text(
                          _alumno!.nombre,
                          style: const TextStyle(fontWeight: FontWeight.bold),
                        ),
                        subtitle: Text('Boleta: ${_alumno!.boleta}'),
                      ),
                      const Divider(),
                      Row(
                        mainAxisAlignment: MainAxisAlignment.spaceBetween,
                        children: [
                          Text('Promedio General: ${_alumno!.promedio}'),
                          Chip(
                            label: Text(
                              _alumno!.esRegular ? 'REGULAR' : 'IRREGULAR',
                            ),
                            backgroundColor: _alumno!.esRegular
                                ? Colors.green.shade100
                                : Colors.orange.shade100,
                          ),
                        ],
                      ),
                    ],
                  ),
                ),
              ),
            ],
          ),
        ),
      );
    }
  }
```

Prácticas Guiadas / Laboratorio

Práctica 1: Creación de un Servicio Local de Caché con *Local DataSource*

1. Agrega una interfaz `AlumnoLocalDataSource` dentro de la capa `Data` para almacenar objetos `AlumnoModel` en memoria o almacenamiento local.
2. Modifica la implementación `AlumnoRepositoryImpl` para implementar el patrón de estrategia: verificar primero la existencia del alumno en el `LocalDataSource` y, únicamente si no existe, invocar al `RemoteDataSource`.

Práctica 2: Adición de un Nuevo Caso de Uso (*Use Case*) para Registro de Alumnos

1. Diseña un caso de uso `RegistrarAlumnoUseCase` en la capa `Domain` que acepte una `Entity` de tipo `Alumno`.

2. Define la regla de negocio: el promedio inicial no puede ser negativo ni superior a 10.0.
3. Agrega el método correspondiente a la interfaz abstracta `AlumnoRepository` y actualiza la UI para permitir invocar ambos casos de uso de manera desacoplada.

Resumen y Puntos Clave

- **Independencia del Dominio:** La capa de Dominio no depende de Flutter, de la UI ni de bases de datos. Es Dart puro conteniendo reglas e identidades de negocio.
- **Inversión de Dependencias:** El Dominio define interfaces abstractas (`AlumnoRepository`), y la capa de Datos se encarga de proveer las implementaciones concretas (`AlumnoRepositoryImpl`).
- **Separación de Entities y Models:** Las *Entities* pertenecen al Dominio y no cambian ante modificaciones en APIs externas; los *Models* pertenecen a Datos y se encargan del parseo de datos (JSON, SQL).
- **Casos de Uso Operativos:** Cada interacción de negocio se representa con un *Use Case* específico, facilitando la creación de pruebas unitarias aisladas.

Bibliografía y Referencias

1. **Martin, R. C.** (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. ISBN: 978-0134494166.
2. **Reso Coder.** (2023). *Flutter Clean Architecture Proposal & Tutorial Series*. Recuperado de: https://resocoder.com/flutter-clean-architecture-cheatsheet
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.